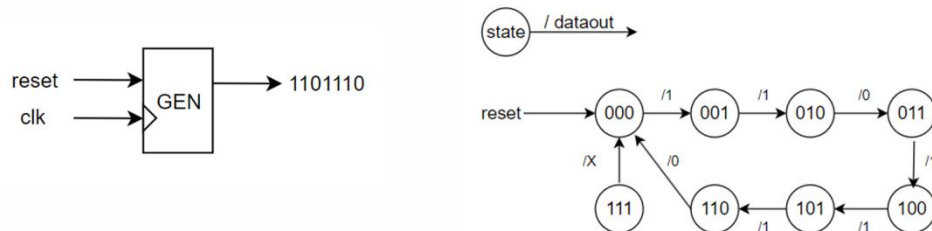


Assignment 2

- 序列信号发生器

按需要产生一组特定串行序列的电路被称为序列信号发生器。下图为“1101110”序列生成器的模块框图和状态转移图。如图所示，发生器有2个输入：第一个是复位信号，用于将状态初始化为000；第二个是时钟输入。



原理：

状态机是一种根据当前状态和输入信号，按照预设规则转换到新状态并产生相应输出的数学模型。时序电路中的存储元件用于保存状态机的当前状态，当输入信号满足转移条件时，状态机根据预设规则更新存储元件中的状态，从而实现时序电路的功能。

VHDL 代码：

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity sequence_signal_generator is
    Port ( clk : in STD_LOGIC;
          reset : in STD_LOGIC;
          sig : out STD_LOGIC);
end sequence_signal_generator;

architecture Behavioral of sequence_signal_generator is
    signal state : unsigned(2 downto 0) := "000";
    signal state_next : unsigned(2 downto 0);
begin
    -- 状态方程
    process (clk, reset)
    begin
        if (reset = '1') then
            state <= "000";
        elsif (rising_edge(clk)) then
            state <= state_next;
        end if;
    end process;
end Behavioral;
```

```

-- 驱动方程
process (state)
begin
    case state is
        when "110" =>
            state_next <= "000";
        when "111" =>
            state_next <= "000";
        when others =>
            state_next <= state + 1;
    end case;
end process;

```

```

-- 输出方程
process(state)
begin
    case state is
        when "000" =>
            sig <= '1';
        when "001" =>
            sig <= '1';
        when "010" =>
            sig <= '0';
        when "011" =>
            sig <= '1';
        when "100" =>
            sig <= '1';
        when "101" =>
            sig <= '1';
        when "110" =>
            sig <= '0';
        when others =>
            sig <= 'X';
    end case;
end process;

```

end Behavioral;

testbench 代码:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity sequence_signal_generator_tb is
-- Port ( );

```


描述和分析:

VHDL 代码:

为了实现状态机的功能，我们需要定义一个 3 位无符号整数作为状态寄存器，用于表示和储存当前状态。

在状态方程部分，我们主要描述状态寄存器如何在时钟触发时更新状态。当复位信号有效时，将状态初始化为“000”；在时钟上升沿触发时，状态寄存器更新为下一个状态。

在驱动方程部分，我们需要定义输出信号如何根据当前状态生成。大部分状态按顺序递增，而当状态为“110”或“111”时，跳转回初始状态“000”，形成循环。

在输出方程部分，输出信号根据当前状态直接生成，通过`case`语句将每个状态映射到特定的输出值。例如状态“000”和“001”时输出为‘1’，状态“010”和“110”时输出为‘0’。

Post-implementation simulation 波形图:

在复位信号变为低电平后，输出信号开始按照预定的逻辑变化，生成“1101110”序列的波形。波形图中输出信号的变化与状态转换图中定义的状态转移及输出映射关系一致，例如在状态“000”和“001”时输出为“1”，在状态“010”和“110”时输出为“0”。

结论:

仿真波形图中的信号变化与状态转换图一致，电路功能正常。

Assignment 3

• 101序列检测器

以下为序列检测器的模块框图和状态转移图。如图所示，检测器有3个输入：

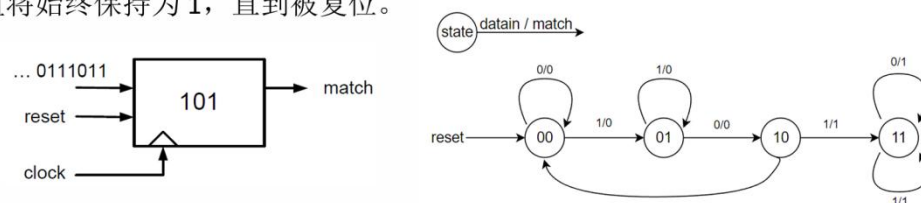
第一个是复位信号，用于将状态初始化为00；

第二个是位串行输入；

第三个是时钟输入。

检测器能够识别输入序列中的 101 序列，并将输出值设置为 1。

输出值将始终保持为 1，直到被复位。



原理:

在本实验中，我们需要设计一个状态机，用于检测特定的输入信号序列。状态机通过状态寄存器保存当前状态，并在时钟信号的上升沿触发时更新状态，复位信号用于将状态初始化。状态转移逻辑通过组合逻辑实现，根据当前状态和输入信号决定下一个状态。输出信号则根据当前状态和输入信号直接生成。

VHDL 代码:

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
```

```
entity sequence_detector is
```

```
    Port ( reset : in STD_LOGIC;
          input_sig : in STD_LOGIC;
          clock : in STD_LOGIC;
          match : out STD_LOGIC);
```

```
end sequence_detector;
```

```
architecture Behavioral of sequence_detector is
```

```
    signal state : unsigned(1 downto 0) := "00";
```

```
    signal next_state : unsigned(1 downto 0) := "00";
```

```
begin
```

```
-- 状态方程
```

```
process(clock, reset)
```

```
begin
```

```
    if (reset = '1') then
```

```
        state <= "00";
```

```
    elsif (rising_edge(clock)) then
```

```
        state <= next_state;
```

```
    else
```

```
        state <= state;
```

```
    end if;
```

```
end process;
```

```
-- 驱动方程
```

```
process(state, input_sig)
```

```
begin
```

```
    if (std_logic_vector(state) = "00" and input_sig = '0') then
```

```
        next_state <= "00";
```

```
    elsif (std_logic_vector(state) = "00" and input_sig = '1') then
```

```
        next_state <= "01";
```

```
    elsif (std_logic_vector(state) = "01" and input_sig = '1') then
```

```
        next_state <= "01";
```

```
    elsif (std_logic_vector(state) = "01" and input_sig = '0') then
```

```
        next_state <= "10";
```

```
    elsif (std_logic_vector(state) = "10" and input_sig = '1') then
```

```
        next_state <= "11";
```

```
    elsif (std_logic_vector(state) = "10" and input_sig = '0') then
```

```
        next_state <= "00";
```

```
    elsif (std_logic_vector(state) = "11" and input_sig = '0') then
```

```
        next_state <= "11";
```

```
    elsif (std_logic_vector(state) = "11" and input_sig = '1') then
```

```

        next_state <= "11";
    end if;
end process;

-- 输出方程
process(state)
begin
    if (std_logic_vector(state) = "00" and input_sig = '0') then
        match <= '0';
    elsif (std_logic_vector(state) = "00" and input_sig = '1') then
        match <= '0';
    elsif (std_logic_vector(state) = "01" and input_sig = '1') then
        match <= '0';
    elsif (std_logic_vector(state) = "01" and input_sig = '0') then
        match <= '0';
    elsif (std_logic_vector(state) = "10" and input_sig = '1') then
        match <= '1';
    elsif (std_logic_vector(state) = "10" and input_sig = '0') then
        match <= '0';
    elsif (std_logic_vector(state) = "11" and input_sig = '0') then
        match <= '1';
    elsif (std_logic_vector(state) = "11" and input_sig = '1') then
        match <= '1';
    end if;
end process;
end Behavioral;

```

testbench 代码:

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sequence_detector_tb is
-- Port ( );
end sequence_detector_tb;

architecture Behavioral of sequence_detector_tb is
component sequence_detector is
    Port ( reset : in STD_LOGIC;
          input_sig : in STD_LOGIC;
          clock : in STD_LOGIC;
          match : out STD_LOGIC);
end component;

signal res_i, sig_i, clk_i, match_o : std_logic := '0';
constant halfperiod : time := 10ns;

```

```

begin
    uut: sequence_detector port
        map( reset => res_i,
            input_sig => sig_i,
            clock => clk_i,
            match => match_o);

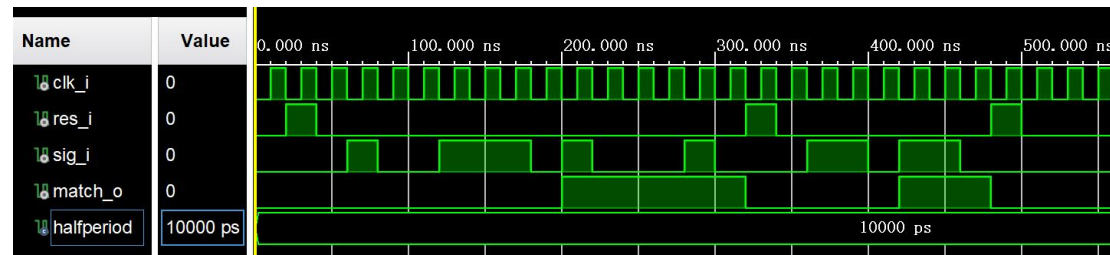
    clk_i <= not clk_i after halfperiod;
    res_i <= '1' after halfperiod*2, '0' after halfperiod*4,
        '1' after halfperiod*32, '0' after halfperiod*34,
        '1' after halfperiod*48, '0' after halfperiod*50;
    sig_i <= '1' after halfperiod*6, '0' after halfperiod*8,
        '1' after halfperiod*12, '0' after halfperiod*18,
        '1' after halfperiod*20, '0' after halfperiod*22,
        '1' after halfperiod*28, '0' after halfperiod*30,
        '1' after halfperiod*36, '0' after halfperiod*40,
        '1' after halfperiod*42, '0' after halfperiod*46;

end Behavioral;

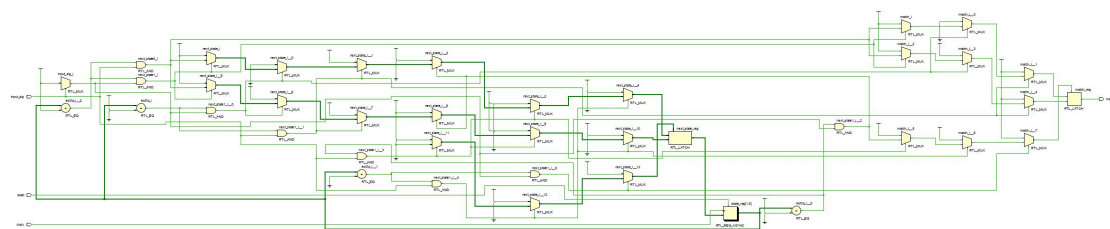
```

实验结果:

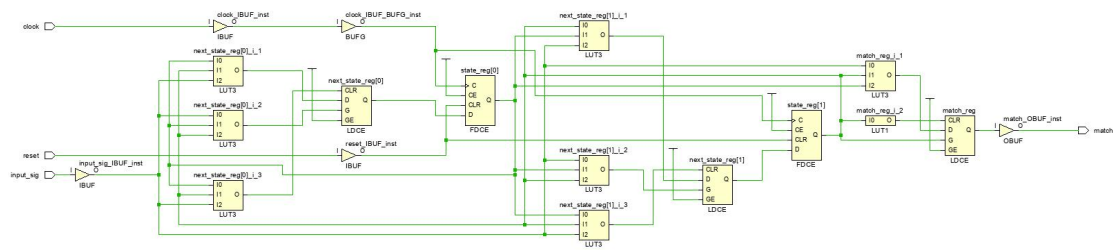
Post-implementation simulation:



RTL schematic:



Implementation schematic:



描述和分析：

VHDL 代码：

为了实现状态机的功能，我们需要定义一个 2 位无符号整数作为状态寄存器，用来表示和储存当前状态。

在状态方程部分，我们主要描述状态寄存器如何在时钟触发时更新状态。当复位信号有效时，将状态初始化为“00”；在时钟上升沿触发时，状态寄存器更新为下一个状态。

在驱动方程部分，我们需要定义输出信号如何根据当前状态和输入信号生成。状态转移逻辑通过 if-elsif 语句实现，根据当前状态和输入信号决定下一个状态。

在输出方程部分，输出信号根据当前状态和输入信号直接生成，通过 if-elsif 语句将每个状态和输入信号组合映射到特定的输出值。例如，当状态为“10”且输入信号为“1”时，输出为“1”。

Post-implementation simulation 波形图：

在复位信号变为低电平后，输出信号开始根据输入信号和状态转移逻辑变化，当检测到特定输入序列“101”之后，输出信号 match_o 被置为 1，并且在复位之前保持不变。波形图中输出信号的变化与状态转移图中定义的状态转移及输出映射关系一致，例如在状态“10”且输入信号为“1”时，输出为“1”。

结论：

仿真波形图中的信号变化与预想设计功能及状态转移图一致。状态机能够根据输入信号正确更新状态，并检测特定的输入序列“101”，生成预期的输出信号，电路功能正常。